

MODFuzz: A Multiobjective Directed Fuzzer for USB Drivers

Yilin Zhou^{1b}, Guojun Peng^{1b}, Xingliang Wang, Chenyang Wang^{1b}, Zichuan Li^{1b}, Side Liu^{1b}, Yanhao Wang^{1b}, Xiuzhang Yang, and Jianming Fu^{1b}, *Member, IEEE*

Abstract—USB interfaces have become ubiquitous in various Internet of Things (IoT) devices, all adhering to the same universal serial bus (USB) protocol. While enhancing convenience, they also widen the potential attack surface. Fuzzing is a proactive way to identify potential security threats for USB drivers. However, existing USB driver fuzzers primarily prioritize the code coverage of USB drivers, leading to a significant waste of computational resources on irrelevant code segments. To this end, we combine directed fuzzing and USB driver fuzzing for the first time, and present multiobjective directed fuzzer (MODFuzz), a pioneering multiobjective directed fuzzing method for USB drivers. MODFuzz autonomously locates the most vulnerable parts within USB drivers, concentrating fuzzing efforts on these areas. Diverging from the existing directed fuzzers, MODFuzz employs a dynamic direction instead of predetermined addresses to guide the fuzzing campaign toward the triggered execution traces with a greater probability of containing vulnerabilities. MODFuzz outperforms the strong baseline in terms of execution speed (about 14% improvement) and crash generation capabilities (about 69% improvement). Meanwhile, we found six previously unknown bugs (all confirmed and assigned vulnerability IDs) in Linux kernel v6.4.10 and received acknowledgment from Red Hat.

Index Terms—Coverage, directed fuzzing, Linux kernel, universal serial bus (USB).

I. INTRODUCTION

THE UNIVERSAL serial bus (USB) protocol specifies the physical, electrical, and communication standards for connecting devices [1]. USB interfaces are pervasive and found in a wide range of devices, from small Internet of Things (IoT) devices to expansive cloud computing servers. While traditional USB connections involve physical contact,

innovative technologies like USBIP [2], [3] and USBRedir [4] facilitate remote usage of the USB protocol over a network.

The widespread use of the USB interface makes it an important target for many attackers. For example, USB drivers are typically developed based on standard behavior paradigms. Developers consider more about error handling rather than security, which creates an opportunity for attackers to craft malicious USB devices capable of executing malicious code [5], [6] or exploiting driver vulnerabilities [7]. Finding unknown vulnerabilities proactively in USB drivers and fixing them in time can mitigate this potential security risk. Fuzzing has been proven to be an effective method for vulnerability mining. From the perspective of seed selection strategy, the existing fuzzers can be divided into: 1) coverage-guided fuzzing [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18] and 2) directed fuzzing [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32]. Coverage-guided fuzzing uses code coverage as the main indicator to measure the quality of a seed. Directed fuzzers try to make the execution traces triggered by the seeds more concentrated on the predetermined parts of the target.

Since the USB drivers run in the OS kernel space, it is not optimal to use coverage-guided fuzzing on it. On the one hand, there are lots of interrupts and timer-based tasks in the kernel, which makes the coverage collection for a single thread in the kernel inaccurate. Hotplugging of a USB device is an asynchronous event. When a USB device triggers code execution in multiple different contexts, it often results in a larger coverage result, which greatly interferes with fuzzing. Peng and Payer [7] alleviated this problem by saving the execution contexts on the stack before calling `do_IRQ`, but this method is not applicable in the latest kernel versions. On the other hand, Wang et al. [33] indicated that different parts of the program have different importance for vulnerability mining. The size of the kernel is large (nearly 200 000 functions in Linux kernel v6.4.10), but the proportion of drivers that really have fuzzing value is small. Theoretically, that makes directed fuzzers more suitable for focusing fuzzing attention on specific USB drivers. As one of the most popular directed fuzzers, the distance-based directed fuzzer was proposed for exploiting or reproducing vulnerabilities in code parts specified in advance [19], [22], [23], [25], [27], [28]. They are suitable for fuzzing predetermined parts of the program. However, the existing distance-based directed fuzzers can not be used directly to fuzz USB drivers (see C1–C3 in Section II-C).

The challenges of existing works motivate us to develop a new directed algorithm that can automatically allocate

Received 17 July 2024; revised 6 November 2024 and 30 January 2025; accepted 25 February 2025. Date of publication 28 February 2025; date of current version 9 June 2025. This work was supported in part by NSFC under Grant 62172308, Grant U1626107, Grant 61972297, Grant 62172144, and in part by Guizhou Provincial Basic Research Program under Grant MS[2025]686. (Corresponding author: Guojun Peng.)

Yilin Zhou, Guojun Peng, Xingliang Wang, Chenyang Wang, Side Liu, and Jianming Fu are with the Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, and the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: ylzhou@whu.edu.cn; guojpeng@whu.edu.cn; xiliwang@whu.edu.cn; wcy62@whu.edu.cn; sidelau@whu.edu.cn; jmfu@whu.edu.cn).

Zichuan Li is with the System Security Foundations Laboratory, Indiana University Bloomington, Bloomington, IN 47405 USA (e-mail: lizic@iu.edu).

Yanhao Wang is with the Department of Security Research, NIO Inc., Beijing 201804, China (e-mail: wangyanhao136@gmail.com).

Xiuzhang Yang is with the Guizhou Big Data Academy, Guizhou University, Guiyang 550025, China (e-mail: xzyang@gzu.edu.cn).

Digital Object Identifier 10.1109/IIOT.2025.3546664

fuzzing's attention to the most vulnerable parts. In our algorithm, the indicators of fuzzing are no longer predetermined targets but a large set of objective functions with their own vulnerability scores (see Section IV-B). The direction of fuzzing is no longer toward the closest path to the fixed targets but to trigger the execution trace that can maximize the sum of included objective functions' vulnerability scores.

We present MODFuzz, a multiobjective directed Fuzzer for USB drivers (the meaning of multiobjective is explained in Section IV-C2). MODFuzz takes advantage of directed fuzzing and concentrates fuzzing resources into a relatively small number of noteworthy USB driver functions in the large kernel space. MODFuzz evaluates the vulnerability scores of each objective function from the perspectives of distance-based function fragility analysis and symbolic execution-based global variable riskiness analysis. The vulnerability score is then used to measure the probability that the corresponding objective function contains vulnerabilities. To make our analysis more precise, we propose the concept of edge transition probability (ETP) to resolve the inconsistency between the distance calculated by the existing algorithm and the actual kernel situation. At last, we use the multiobjective directed fuzzing algorithm to guide the fuzzing campaign toward the triggered execution traces with higher vulnerability scores.

We choose the Linux kernel as our research target for two reasons. The first reason is that from the perspective of IoT device market share, although there are various systems used by IoT devices, the latest statistics [34] show that Linux-based systems have the largest market share (more than 68% and still increasing), far exceeding other operating systems such as FreeRTOS, etc. As for the second reason, since Peng and Payer [7] have shown similarities in USB driver logic across multiple operating systems through cross-pollination experiments, we implemented MODFuzz for USB driver fuzzing for only Linux system.

Our evaluation shows that our distance calculation method is more accurate than state-of-the-art (SOTA) methods. The results of static vulnerability analysis significantly improve MODFuzz's vulnerability mining capability. MODFuzz outperforms SOTA USB driver fuzzers in terms of execution speed (about 14% improvement to USBFuzz [7]) and crash generation capabilities (about 69% improvement to Syzkaller-USB [35]). After a 72-h fuzzing campaign, we got 49 bugs and finally found six previously unknown bugs in Linux kernel v6.4.10 and reported them.

According to the latest report [36], 51% of malware attacks are designed for USB devices, which is a nearly six-fold increase from 9% reported in the 2019 report. And the emergence of Stuxnet [37] has illustrated the increasing prevalence of vulnerability exploits in USB attacks. The current USB protocol is developing toward greater bandwidth and faster speeds while ensuring forward compatibility, but its core data transmission logic has not been fundamentally changed. MODFuzz is designed to find and fix potential USB framework vulnerabilities in advance, which has long-term positive significance for the security of the USB protocol. In summary, we make the following contributions.

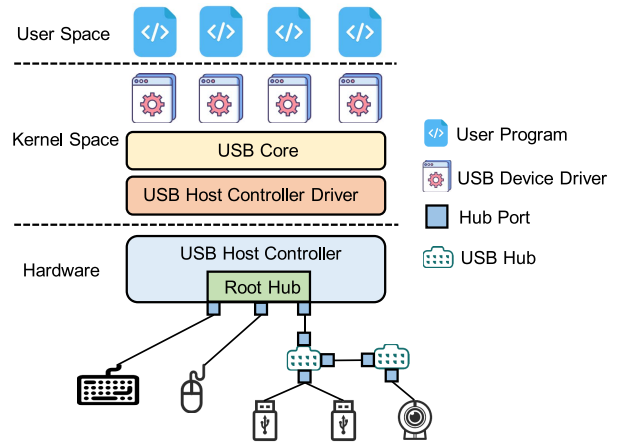


Fig. 1. Architecture of USB driver framework.

- 1) *Key Insights:* By attempting to apply directed fuzzing in USB drivers, we summarize three key insights to guide our research. They point out the three critical issues in directed fuzzing for USB drivers: a) vulnerability regularity for functions; b) binary distance calculation method; and c) local optimal problem.
- 2) *Innovative Algorithm:* We first propose a multiobjective directed algorithm for USB driver fuzzing. Compared with the traditional directed fuzzing strategy, our fuzzing direction changes to a dynamic and macroscopic way, which is to maximize the sum of the vulnerability scores of the triggered execution trace rather than predetermined fixed addresses.
- 3) *Open-Source Implementation:* We implement and release the code of MODFuzz [38], which is able to fuzz all versions of Linux kernels with minor automated patches.
- 4) *Experimental Results:* Our experiments show that MODFuzz outperforms SOTA USB driver fuzzers (i.e., Syzkaller-USB and USBFuzz) in execution speed and crash generation capabilities. It also outperforms SOTA distance-based directed fuzzing algorithm (i.e., algorithms for Hawkeye and AFLGo) in terms of the time to exposure (TTE). We discovered six previously unknown bugs in Linux kernel v6.4.10 and received acknowledgment from Red Hat.

II. BACKGROUND AND RELATED WORK

A. USB Driver Framework

The USB protocol [39] has gone through numerous iterations in its more than 20 years of development, all of which share a common high-level architecture. Fig. 1 shows the client-server architecture of the USB driver framework. From a topological point of view, multiple USB devices are connected to the root hub through USB hubs to form a tree structure. USB host controller driver (HCD) abstracts the USB host controller into a software layer for use by the upper USB core. The USB core implements functions such as hot plug control of USB devices and data bus transmission control. Then, the USB device drivers can directly use the interfaces

provided by the USB core to implement corresponding functions (e.g., mouse and keyboard control, mass storage, etc.). Different layers in the USB driver framework have different responsibilities. It is necessary for a sound USB driver fuzzer to cover all of them.

B. USB Driver Fuzzing

USB driver fuzzing means providing malformed input to USB drivers, causing the kernel thread that is handling the USB device to crash. There have been some representative fuzzers for USB drivers [7], [35], [40], [41], [42], [43]. The fuzzing for USB drivers has experienced the evolution from relying on hardware to pure software implementation (e.g., system emulation by QEMU [44]).

Fuzzing USB Drivers With Hardware Assistance: van Tonder and Engelbrecht [41] enabled man-in-the-middle USB fuzzing between a host and peripheral. But it is difficult to expand on a large scale due to their dependence on customized hardware. FUZZPD [40] focuses on the security of USB power delivery (USBPD) firmware and tests the charging protocol, but its requirement for a physical USB Type-C connection makes it not scalable.

Fuzzing USB Drivers Based on Emulation: As a hardware-independent kernel fuzzer, Syzkaller [35] also supports fuzzing for USB drivers. But its special USB data injection mechanism (by external pseudo-syscalls) makes it too dependent on manual modification of different kernel versions based on expert knowledge. ReUSB [42] improves code coverage by performing replay at the USB driver boundary. FuzzUSB [43] performs USB fuzzing by extracting internal state machines from USB gadget drivers. It focuses on fuzzing USB gadget rather than USB host. Both ReUSB and FuzzUSB are implemented based on Syzkaller, so they also involve some expert efforts. USBFuzz [7] emulates the plugging and unplugging process of USB devices through pure software implementation, thus achieving coverage-based fuzzing without modifying the USB system code in the kernel.

However, these fuzzers all use code coverage as an essential indicator. Coverage-guided fuzzing, although achieving many amazing results, is not the most suitable method for USB driver fuzzing. Interferenced by a large number of interrupts and timer-based tasks in kernel space, coverage collection will be inaccurate. Different parts of a driver are also not equally important. And the code that may cause vulnerabilities only accounts for a small part [33]. So the coverage can not reasonably measure the effect of fuzzing. Distance-based directed fuzzing is a more efficient option for fuzzing the fragile code in the drivers. However, to the best of our knowledge, there is no published work applying distance-based directed fuzzing to USB drivers.

C. Directed Fuzzing and Distance-Based Directed Fuzzing

Unlike coverage-guided fuzzing that pursue broad coverage, directed fuzzing [19], [20], [22], [25], [27], [28], [45] often have a clear fuzzing direction. For example, UAFUZZ [20] greatly improves the efficiency of finding use-after-free (UAF) vulnerabilities by customizing the UAF execution feature as

TABLE I
VULNERABILITY TYPES AND ORIGINAL TRIGGER FUNCTIONS
CLASSIFICATION (PARTIAL)

Vulnerability Type	Original Trigger Function
Buffer Overflow	<code>memcpy</code> , <code>strcpy</code> , <code>strcat</code>
Use-After-Free/Double Free	<code>free_pages</code> , <code>kfree</code> , <code>vfree</code>
Privilege Escalation	<code>__sys_setuid</code> , <code>ksys_chroot</code>

the direction of fuzzing. Parmesan [45] uses sanitizer's ability to detect memory operation sequence errors to guide the direction of fuzzing. The directions they choose significantly improve their efficiency in specific scenarios.

Distance-based directed fuzzer (also known as DGF, directed greybox fuzzing) is the earliest and widely used type of directed fuzzing. Its direction is making the execution trace closer to predetermined addresses. Compared with other types of directed fuzzers, its ability to focus on specific code segments makes it more suitable for custom fuzzing of drivers at various levels in the USB architecture. However, in the scenario of USB driver fuzzing, the existing works [19], [25], [27], [28] still face some challenges (C1–C3).

- C1: Distance-based directed fuzzing needs to manually pre-determine the targets. However, from the perspective of vulnerability mining, the predetermined targets are not always the most objectively valuable ones. Lack of a solution for how to choose valuable targets among the huge number of kernel functions automatically (see Section IV-C).
- C2: When calculating the binary distances, the existing algorithms only consider the number of edge between functions or basic blocks. They are inaccurate in the more complex kernel driver scenarios (see Section IV-B3).
- C3: Existing instrumentation scheme of distance-based directed fuzzing, which writes distance information into instrumentation data, can only support fixed targets and has poor scalability. When the targets change, the reinstrumentation and recompilation are required. Lack of a flexible instrumentation method for multiple kernel versions and different fuzzing parameters (see Section IV-A).

III. KEY INSIGHTS

A. Regularity of USB Driver Vulnerability

After manually analyzing 197 USB driver-related publicly disclosed vulnerabilities (got by keyword matching) in CVE [46], we summarize the regularity of USB driver vulnerability triggering by the following three observations.

- 1) While the potential combinations of functions that can lead to kernel vulnerabilities are virtually limitless, the specific functions that ultimately trigger these vulnerabilities are finite and identifiable. For example, there are more than 15 000 buffer overflow vulnerabilities in CVE [46], but the common point of the code that triggers these vulnerabilities is that they all inevitably call a few basic functions such as `memcpy` or `strcpy`.

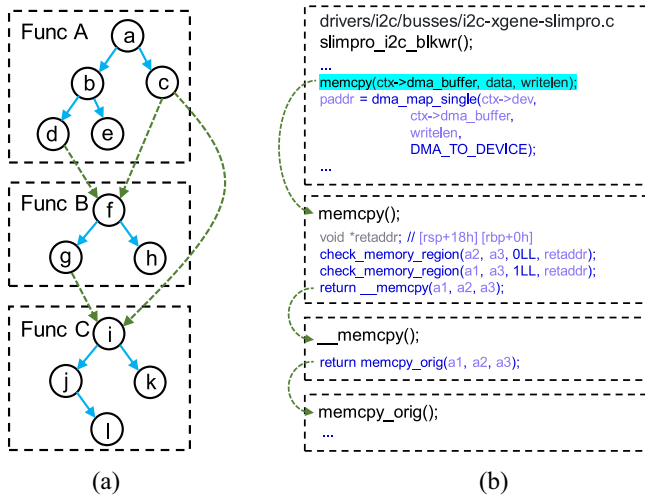


Fig. 2. Function call chains in two different scenarios. Circles represent basic blocks, dashed boxes represent functions, blue solid arrows represent transitions between basic blocks, and green dotted arrows represent function calls. The division of basic blocks is in the style of IDA Pro [47]. (a) Abstract function call scenario. (b) Real-world function call scenario in CVE-2023-2194 [48].

In Table I, we list some common vulnerability types and their original trigger functions. To make MODFuzz work better, we also built a more complete set of original trigger functions based on the vulnerability information disclosed by CVE.

- 2) Improper use of global variables is one of the most important parts of many complex vulnerabilities and complex attack vectors. If a global variable is modified to a dangerous value, it may cause unexpected behavior in other normal functions that reference it. In USB driver architecture, the values of many structure members are read from the USB device and used as global variables by many USB drivers. Therefore, references to these global variables can be seen as potential factors leading to vulnerabilities.
- 3) USB-related functions refer to the functions in the kernel related to the USB device processing procedure. Since our ultimate goal is to fuzz the USB driver framework, we should pay more attention to USB-related functions during static analysis (according to our analysis, USB-related functions only account for about 15% in the Linux kernel). The smaller the distance between a function f and USB-related functions, the higher the degree of attention f should receive.

We evaluate a function's vulnerability score based on these observations in static vulnerability analysis (see Section IV-B6).

B. Inaccurate Distance Calculation Method

In distance-based directed fuzzing, the accuracy of distance calculation is crucial. However, the SOTA works (i.e., AFLGo and Hawkeye) are not suitable for the kernel fuzzing scenario.

Limitations of AFLGo: When calculating the distance between two sites, AFLGo takes the edge number of the shortest path between them [19]. In Fig. 2(a), when calculating the

distance from a to i , due to the existence of path $\langle a, c, i \rangle$, AFLGo will use the length of $\langle a, c, i \rangle$ as the distance result. The paths $\langle a, b, d, f, g, i \rangle$ and $\langle a, c, f, g, i \rangle$ are discarded because they are longer. Since the binary distance is used to measure the difficulty from a to i , the existence of these two longer paths should make the distance smaller because they increase the probability of reaching i from a during the actual execution process.

Limitations of Hawkeye: Hawkeye is improved on the basis of AFLGo, taking into account the $\Phi(C_N) = ([\phi \cdot C_N + 1]/[\phi \cdot C_N])$ (C_N is the number of callee site occurrences while ϕ is a constant) and $\Psi(C_B) = ([\psi \cdot C_B + 1]/[\psi \cdot C_B])$ (C_B is the number of basic blocks that contains at least one call site of the callee while ψ is a constant), and using their product as the distance between the two sites [25]. Hawkeye can achieve more reasonable results in Fig. 2(a). But in the case of Fig. 2(b), both AFLGo and Hawkeye are not accurate. There are many simple wrapper functions in the Linux kernel, such as the frequently used `memcpy`. If a driver uses `memcpy` in its source code, the execution trace in the compiled kernel binary will go through the process of `memcpy` $\rightarrow$ `__memcpy` $\rightarrow$ `memcpy_orig`. In this procedure, except for `memcpy_orig`, which is the function that actually implements the memory copy operation, `memcpy` and `__memcpy` are called wrapper functions. They are as simple as only one basic block without any branches. That means that if `slimpro_i2c_blkwr` is called in Fig. 2(b), `memcpy_orig` will be called with 100% probability. The distance between them should be a relatively small value. However, both AFLGo and Hawkeye will get significantly larger distance results, which ultimately reduce the efficiency of directed fuzzing. We propose the ETP to improve the defects of existing methods (see Section IV-B3).

C. Local Optimum Problem for Directed Fuzzing

If we see coverage-guided fuzzing as pursuing breadth, we can see directed fuzzing as pursuing depth. However, blindly pursuing depth may lead to local optimum problem. The existing directed fuzzers have various types of fuzzing directions. For example, UAFuzz [20] uses the UAF execution feature as the direction and has achieved good results in finding UAF vulnerabilities, but its ability to detect other types of vulnerabilities is weak; traditional distance-based directed fuzzers such as AFLGo and Hawkeye use predetermined addresses as the direction, so it is difficult for them to find vulnerabilities in other code parts, etc. From a global perspective, no matter which fixed fuzzing direction they choose, they will sacrifice the ability to find vulnerabilities outside the direction. While the direction of the directed fuzzer provides various performance improvements, it also makes the fuzzing process fall into a local optimum problem to a certain extent. We propose to use a dynamic direction to guide the fuzzing to alleviate this problem (see Section IV-C2).

IV. DESIGN AND IMPLEMENTATION

We design the MODFuzz framework as shown in Fig. 3. Overall, we first preprocess the source code of the target kernel

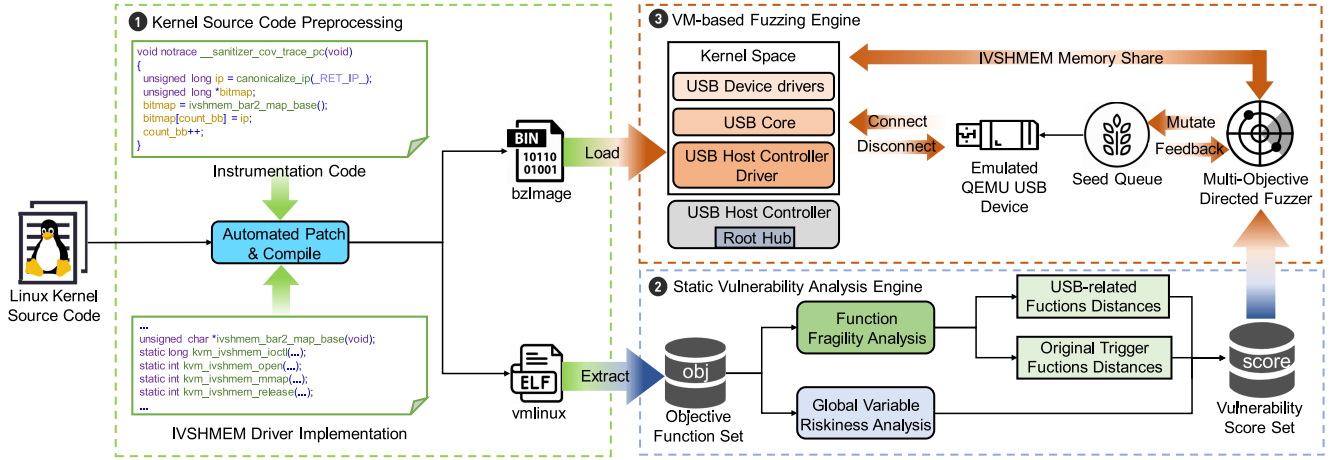


Fig. 3. MODFuzz overview.

and conduct static vulnerability analysis to obtain the vulnerability score of each objective function. The vulnerability score set is then used to guide the VM-based fuzzing engine to perform multiobjective directed fuzzing for USB drivers.

A. Kernel Source Code Preprocessing

Preprocessing of the kernel source code is necessary for kernel emulation and fuzzing. It consists of the following steps.

Lightweight Instrumentation: The purpose of instrumentation for MODFuzz is to collect all the function names in the execution trace with minimal performance overhead. Kcov [49] is a popular open-source kernel source code instrumentation solution, which has been integrated into the Linux kernel source code since v4.6. But it is aimed at collecting coverage, and the time complexity is relatively high due to lots of array and hash operations. We ameliorate kcov to adapt to our multiobjective directed fuzzing algorithm. The instrumentation function will be inserted at the end of each basic block. We remove the code that collects coverage and use the macro `_RET_IP_` in the instrumentation function to obtain the address inside the currently executing function. This address is then compared with the function boundary provided by static analysis to obtain the name of the executing function. Compared with coverage-guided style instrumentation (original kcov), the size of the compiled binary code of our instrumentation function is reduced by 66% (from 353 bytes to 120 bytes). In Section V-C, we demonstrate the performance improvement brought by this. Compared with distance-based directed instrumentation for fixed addresses, our instrumentation solution only records function names in the execution trace, so there is no need to recompile the kernel after changing the objective function set information.

Instrumentation Scope Customization: There are nearly 200 000 functions in the Linux kernel v6.4.10, but only about 15% of the functions are related to the processing of USB devices. Instrumenting all kernel functions is impractical and unnecessary. Our goal is fuzzing the USB driver framework, not the entire kernel space. Therefore, we only instrument the basic Linux functions (i.e., functions like `memcpy` that implement basic system operations) and the functions related

to USB. To balance performance and availability, we do not instrument the functions that will cause significant emulation overhead (e.g., basic I/O functions). The instrumentation scope of MODFuzz for Linux kernel v6.4.10 is shown in Table V (see the Appendix). The instrumentation scope can also be customized as needed when fuzzing specific drivers is required.

IVSHMEM Driver Implementation: We use IVSHMEM [50] (Inter-VM shared memory device) to implement memory exchange between the fuzzer and the fuzzed VM. Specifically, we add it to `drivers/staging/` in the kernel source code directory and register IVSHMEM as an emulated PCI hardware device in QEMU.

B. Static Vulnerability Analysis

We perform static analysis mainly based on compiled binaries (i.e., `vmlinux`) instead of the source code. Due to the large number of compilation options provided by the Linux kernel, there is a huge semantic gap between the source code and the compiled binary. The same source code will result in very different binary files under different compilation options. Only the information in the compiled binary can accurately restore the actual execution situation [51], [52]. The `bzImage` obtained after kernel compilation is small in size (about 35 MB in v6.4.10) and can be used for emulation. It is a compressed bootable kernel, but it is difficult to perform static analysis without a symbol table. While `vmlinux` is an intermediate product of the compilation process, which is larger (about 240 MB) and can not be executed directly. But it contains rich debugging information. Since every function address and function name of `bzImage` and `vmlinux` compiled from the same kernel source code are consistent, we perform static vulnerability analysis on `vmlinux`.

1) Objective Function Set: Objective functions refer to the set of functions that need to be calculated for vulnerability scores in static vulnerability analysis. Theoretically, the most accurate execution path evaluation result can only be obtained by treating all the instrumented functions in the entire kernel except nonroot nodes in the wrapper function chains as objective functions and calculating their vulnerability scores.

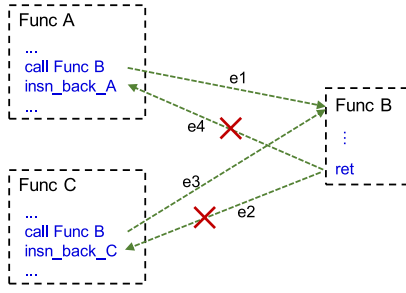


Fig. 4. Example of ICFG reconstruction. The red cross represents the edge that needs to be pruned.

However, the selection of the objective function set needs to balance the metrics of efficiency and accuracy. More functions participating in the calculation as objective functions will result in more accurate execution path vulnerability score results, but the performance overhead will also be higher. From the distribution of vulnerability scores in Fig. 9, it can be seen that objective functions with high vulnerability scores do not account for a large proportion of the entire objective function sets. Therefore, the selection of the objective functions should cover the core functions of the USB framework and functions with high vulnerability scores as much as possible, while trying to exclude functions that are not related to the USB framework (these functions accounts for the vast majority of kernel functions and will significantly reduce efficiency). This is our way to optimize the calculation efficiency while minimizing the negative impact on accuracy. Specifically, we build the objective function set in the following two ways.

Through Execution Trace: By using our instrumentation solution in Section IV-A, we collect driver function names called during USB device interaction processes multiple times. This allows the core functions in the USB framework to be included.

Through Source Code: We use keyword-based regular expressions to select functions related to the USB framework in the Linux kernel driver source code. This ensures that functions related to the USB framework but with low actual execution frequency are not missed.

The execution trace contains dynamic information, and the source code contains static information. By taking the union of them, we get the final objective function set S_{OF} .

2) *ICFG Reconstruction:* The development of the static vulnerability analysis engine is mainly based on angr [52], [53]. Constructing an accurate interprocedural control-flow graph (ICFG) is critical to the plausibility of our analysis. However, due to the existence of a large number of callback functions, the `vmlinux` compiled by the Linux kernel is essentially a nonfully connected graph from a binary perspective. Angr can not generate high-precision ICFG through symbolic execution. Therefore, the ICFG generated entirely through static analysis needs to be pruned and reconstructed.

As shown in Fig. 4, for a function call from Func A to Func B, since the call to B will eventually return to the next instruction (we mark this instruction as `insn_back_A`) of the call instruction in A, angr adds an edge (e4 in Fig. 4) between the base block to which `insn_back_A` belongs

and B in ICFG. This added edge is reasonable to a certain extent. However, if another Func C also calls B, it will cause a path (`<e3, e4>`) from C to the basic block to which the `insn_back_A` in A belongs from the ICFG perspective. Even there is no theoretical path between C and A. The existence of such misleading edges (e.g., e3 and e4) greatly affects the accuracy of static analysis. Therefore, we prune this type of edge when reconstructing ICFG to avoid similar situations.

3) *Edge Transition Probability:* Inspired by the application of path probability [54], [55], [56] to symbolic execution optimization, we propose the ETP to measure the binary distance between two sites to solve the inaccuracy of distance calculation proposed in Section III-B. For the start site x and the destination site y , we calculate the ETP between them as follows: 1) obtain all paths in ICFG from x to y without loops, and record their number as N_p ; 2) for each path i , record the number of its nodes as $N_b(i)$; and 3) for each nonleaf node j on each path, get its successors' number $suc(i, j)$, and (iv) calculate the ETP $ETP(x, y)$ by (1). Then, we can use $ETP(x, y)$ to measure the binary-level distance between site x and y , which can be a basic block or a function. When x or y is a basic block, we directly use it as the start or destination site. If x or y is a function, we take its first basic block into calculation instead

$$ETP(x, y) = \sum_{i=0}^{N_p} \prod_{j=0}^{N_b(i)} \frac{1}{suc(i, j)}. \quad (1)$$

Our design idea is to consider each basic block transfer process of all paths between two sites based on branch probabilities. The sum operation in (1) ensures that ETP considers all the paths from the start site x to the destination site y , not just the shortest path. The existence of each new path (regardless of how many basic blocks it passes through) will probabilistically shorten the distance from x to y , which solves the problem that traditional algorithms (e.g., AFLGo's algorithm) get larger distance calculation results for two sites with multiple paths between [example in Fig. 2(a)]. At the same time, the product operation in (1) ensures that ETP considers every branch in a path when calculating each path. The existence of each branch in the path will reduce the value of ETP, which solves the problem that traditional algorithms (e.g., Hawkeye's algorithm) get larger distance calculation results for wrapper function chains [example in Fig. 2(b)]. Although ETP is calculated based on probability, its value can exceed 1. For instance, when there are two calls to `memcpy()` within a basic block α , the $ETP(\alpha, memcpy_orig)$ is 2. Even if there are additional simple wrapper functions like `__memcpy()` between `memcpy()` and `memcpy_orig()`, they will not affect the value of $ETP(\alpha, memcpy_orig)$. In subsequent calculations, ETP is mainly used as a coefficient to measure binary distance. Therefore, in the two scenarios depicted in Fig. 2, our proposed ETP yields more accurate and reasonable results compared to AFLGo and Hawkeye.

However, our approach, which treats the probability of each branch as the same, is not perfect. In Section VI, we discuss a

possible way (by execution records) to optimize our approach in future work.

4) *Function Fragility*: Function fragility represents the probability that a single USB driver function contains vulnerabilities. For a function f , the entire function fragility $\Omega(f)$ calculation process can be described by

$$\Omega(f) = \sum_{i=0}^{S_{CVE}^{simp}} (ETP(f, f_i)) \cdot \sum_{j=0}^{S_{USB}} (\gamma_j \cdot ETP(f, f_j)). \quad (2)$$

It is calculated based on the ETP of f and each function in the USB correlation function set (S_{USB}) as well as the original trigger function set (S_{CVE}^{simp}). Our basic assumption is: if f itself is vulnerable but has little relevance to the USB framework, then f is not worthy of attention for MODFuzz, and vice versa. So we use multiplication instead of addition to combine them. Specifically, we perform function fragility analysis through the following steps.

- 1) *Original Trigger Function Set Construction*: We first obtain set S_{CVE} by manually extracting information from 197 USB driver-related and 531 kernel-related publicly disclosed vulnerabilities. Considering that calling any function in the wrapper function chain results in the same vulnerability triggering effect (e.g., calling `memcpy` and calling `memcpy_orig` are functionally equivalent), we obtain the wrapper function chain of each function in the set S_{CVE} through static analysis. We only retain the functions at the end of the wrapper function chains and remove all the other functions in the front part of the chains to get the simplified set S_{CVE}^{simp} as the original trigger function set.
- 2) *USB Correlation Function Set Construction*: With our lightweight instrumentation, we can record an execution trace every time the USB device interacts with the kernel driver. By recording over 10 000 real-world USB device interaction execution traces, we obtain the frequency of each function is called. We construct the USB correlation function set S_{USB} , which consists of functions that appear more than 10 times in the recorded execution traces to eliminate interference from accidentally called irrelevant functions. Higher frequency functions in S_{USB} include USB core, USB port operation modules, URB [57] (USB Request Block) processing functions, etc. Lower frequency functions are mainly composed of specific commonly used device drivers, device bus operation modules, probe functions of specific devices, etc. In (2), for a function f_j , the corresponding frequency is γ_j . This allows functions that are more important in the USB driver framework to get higher weight in the calculation.
- 3) *Function Fragility Score Calculation*: For the function to be analyzed, we define it as the source node, then obtain the ETP calculation results of it and each function in S_{USB} as well as S_{CVE}^{simp} . Finally, we obtain its function fragility score by (2).
- 5) *Global Variable Riskiness*: Dangerous global variables are defined as those global variables that have the potential to affect parameters of memory operations or critical logic, and

```
drivers/i2c/busses/i2c-ismt.c
static int ismt_access(..., i2c_smbus_data *data)
{
    ...
    case I2C_SMBUS_BLOCK_PROC_CALL:
        dev_dbg(dev, "I2C_SMBUS_BLOCK_PROC_CALL\n");
        if (data->block[0] > I2C_SMBUS_BLOCK_MAX)
            return -EINVAL;
        dma_size = I2C_SMBUS_BLOCK_MAX;
        desc->tgtaddr_rw = ISMT_DESC_ADDR_RW(addr, 1);
        desc->wr_len_cmd = data->block[0] + 1;
        desc->rd_len = dma_size;
        desc->control |= ISMT_DESC_BLK;
        dma_direction = DMA_BIDIRECTIONAL;
        dma_buffer[0] = command;
        memcpy(&dma_buffer[1], &data->block[1], data->block[0]);
        break;
    ...
}
```

Fig. 5. CVE-2022-3077 [58] related source code snippets. The blue highlight represents the root cause of the vulnerability, and the green highlight represents the code that fixes the vulnerability.

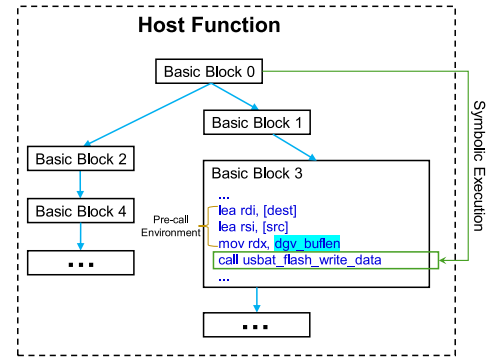


Fig. 6. Symbolic execution-based inspection of the precall environment. The called function f here is `usbat_flash_write_data`.

can cause system crashes when their values deviate from the expected range. We consider references to dangerous global variables as one of the indicators for calculating vulnerability scores. Existing works [59], [60], [61] have demonstrated the criticality of malicious or inappropriate modification of global structure and variables in exploiting Linux kernel vulnerabilities. As shown in Fig. 5, data is passed in as a structure argument, and its member variable `block[0]` is unchecked as the third argument of `memcpy`, causing a buffer overflow.

Unfortunately, USB device information is often passed into the kernel as the global structure which will be processed by multiple driver functions. Our basic idea is: if a dangerous global variable directly serves as an argument of a function f or affects the registers and the values on the stack before calling function f (i.e., indirectly affecting the arguments of function f), then we suppose that the maliciously modification of this argument may cause f to behave undesirably, so we should increase the vulnerability score of function f . Specifically, our global variable riskiness analysis steps are as follows.

- 1) *Dangerous Global Variable Extraction*: There are many global variables in the Linux kernel, but not all global variables may cause risks. We first extract all kernel global variables to form a set S_{gv} according to the permissions and address range of the sections in the `vmlinux` symbol table. Through regular matching of keywords (e.g., `num`, `len`, `buf`, etc.), we get a set S_{dgv}

of global variable names that may potentially cause memory errors.

- 2) *Cross-Reference Set Construction*: For each variable in S_{dgv} , we only care about the read type cross-references. The read type cross-references refer to assembly statements that use global variables as assignment source operands, such as `mov rdx, cs:global_var` or `mov rdx, offset global_var`. The writing operation to dangerous global variables may create potential conditions for triggering the vulnerability. But it will not trigger the vulnerability immediately, which is not helpful for observing destructive phenomena during fuzzing. We mark the cross-reference set that meets the requirements as S_{Xref} .
- 3) *Symbolic Execution-Based Inspection of the Precall Environment*: As shown in Fig. 6, for each cross-reference in S_{Xref} , we find the host function (i.e., where the cross-reference is located) and perform symbolic execution with the beginning of the host function as the start point and the next call assembly statement after the cross-reference as the endpoint. Before symbolic execution, we treat all uninitialized variables as symbols. After symbolic execution ends, according to the parameter passing rules of the 64-bit Linux system [62], we check the values of `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9` and the first 32 bytes of memory on the stack (i.e., precall environment) to see if there is a concrete address pointing to a dangerous global variable. The more addresses pointing to dangerous global variables in the precall environment, the higher the score of the called function f [i.e., $\tau_{dgv}(f)$] in the global variable riskiness analysis. Specifically, we first initialize $\tau_{dgv}(f)$ of each function f to 0. If a concrete address pointing to a dangerous global variable appears in the precall environment of the called function f , then the value of $\tau_{dgv}(f)$ is increased by 1.

Theoretically, the earlier symbolic execution starts, the more accurate the result will be. If symbolic execution starts too late, most of the values in registers and stack will be uninitialized symbols. We finally chose to perform symbolic execution from the start of host function for a balance between performance and usability (avoid path explosion).

6) *Potential Vulnerability Trigger Paradigm*: As shown in (3), inspired by our insight in Section III-A, we propose the potential vulnerability trigger paradigm based on function fragility and global variable riskiness analysis

$$\xi(f) = \Omega(f) \cdot (1 + \tau_{dgv}(f)). \quad (3)$$

In (3), for a kernel function f , $\Omega(f)$ is the function fragility analysis result (see Section IV-B4) and $\tau_{dgv}(f)$ reflects the reference situation to dangerous global variables (see Section IV-B5). $\xi(f)$ represents the vulnerability score calculated by our potential vulnerability trigger paradigm. Consider that not all objective functions have nonzero $\tau_{dgv}(f)$ values in actual calculations, but references to dangerous global variables will significantly increase the risk of f . So we set $\tau_{dgv}(f)$ to the magnification coefficient of $\Omega(f)$.

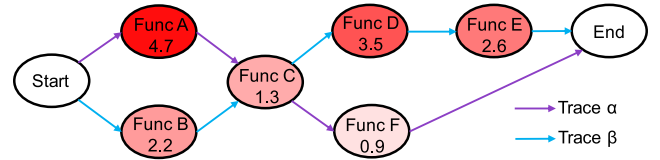


Fig. 7. Two execution traces in abstract scenarios. The number below the function name is the vulnerability score of the function. Arrows of different colors represent different traces.

7) *Vulnerability Score Set*: By performing distance-based function fragility analysis and symbolic execution-based global variable riskiness analysis on the objective function set, we obtain the corresponding vulnerability score set according to (3). It is worth noting that since each function in the wrapper function chain is equivalent, they share the same vulnerability score (e.g., `memcpy` and `memcpy_orig`). As the final analysis result of our static vulnerability analysis, the vulnerability score set is provided as a critical input for subsequent multiobjective directed fuzzing.

C. Multiobjective Directed Fuzzing

1) *Feed the Input to Fuzzer*: How to provide USB input to the driver in the VM's kernel is a crucial issue. There are currently three ways: 1) through real programmable USB devices, such as FaceDancer [63]; 2) by modifying the kernel to inject data into the USB stack through external pseudo-syscalls, such as Syzkaller [35]; and 3) by emulating a USB device in the VM, such as USBFuzz [7]. In order to ensure the adaptability to multiple versions of the kernel, we choose USBFuzz's nonintrusive and scalable software implementation. Specifically, we use an emulated USB device in QEMU and define its actions to connect and disconnect from the VM. Fuzzer and VM share memory through IVSHMEM, but the content of the shared memory is no longer a bitmap recording coverage information. It contains a series of function addresses obtained through `_RET_IP_` to get corresponding function names. In the VM, IVSHMEM is an emulated PCI device, and the corresponding driver is added to the kernel when it is patched. The UMA (User Mode Agent) in the VM is used to continuously read the kernel log file `/dev/kmsg` to obtain kernel crash information and inform IVSHMEM through `ioctl` to start or stop running a test case.

2) *Multiobjective Directed Fuzzing Algorithm*: Different from existing fuzzing algorithms, MODFuzz uses the vulnerability scores of multiple objective functions in the driver under test to dynamically guide the direction of fuzzing, which is the meaning of multiobjective directed. To strike a balance between overhead and performance, our basic idea is just to give seeds that can trigger a higher sum of vulnerability scores more energy (i.e., mutation opportunities and computing resources) and priority to execute. It is easy to locate the function with the highest vulnerability score, but it is difficult to find the execution trace with the highest sum of vulnerability scores. As shown in Fig. 7, trace α has Func A with the highest score (4.7), but its overall score ($4.7 + 1.3 + 0.9 = 6.9$) is lower than trace β ($2.2 + 1.3 + 3.5 + 2.6 = 9.6$). Although the execution trace's vulnerability

score calculation seems to resemble an classical longest path problem in a directed graph, it can not be solved by dynamic programming (DP). The premise of DP is that the global problem can be decomposed into subproblems of multiple stages. However, in fuzzing, the execution trace is determined once the input is determined, and it is almost impossible to modify the execution trace on a arbitrary specified node by customizing the input. Conversely, modifying a single byte in the input may result in a completely different execution path. Therefore, the problem of execution traces triggered by inputs in fuzzing can not be decomposed into multistage subproblems like the travelling salesman problem (TSP). Specifically, our multiobjective directed algorithm is summarized as follows.

- 1) *Execution Trace Parsing*: For each execution trace, the fuzzer collects information fed back by the instrumentation code and gets the objective function names included in this trace within all objective functions.
- 2) *Trace Vulnerability Score Calculation*: For each objective function that appears in this execution trace, get its vulnerability score in the vulnerability score set and accumulate them to get the vulnerability score of this trace finally.
- 3) *Vulnerability Score-Based Seed Mutation*: The seed mutation stage of MODFuzz is similar to AFL, which is also divided into bitflip, arith inc/dec, random havoc, etc. The difference is that AFL primarily assesses seed quality by coverage, while MODFuzz uses the vulnerability scores of execution traces. MODFuzz utilizes a monotonically increasing function $\eta(x)$ (usually an exponential function) to calculate the parameters (e.g., execution time and rounds) in different mutation stages. The x refers to the vulnerability score of the execution trace triggered by the seed. Therefore, in the multiobjective directed fuzzing algorithm, the direction of fuzzing is actually guided by multiple objective functions. This allows seeds that trigger execution traces with higher vulnerability scores to have much more computing resources.

Finding the execution trace with the highest vulnerability score in the huge kernel space is essentially a nondeterministic polynomial problem. We assume that the number of elements in S_{OF} is n . Considering the possible recursion and mutual calls in objective functions, in the worst case, the time complexity of our multiobjective directed fuzzing algorithm is $\mathcal{O}(2^n)$ and the space complexity is $\mathcal{O}(n^2)$. Our algorithm is close to brute force search in terms of computational complexity, but we do not actually need to find the seed that can theoretically get the maximum execution trace vulnerability score. We just make the seed with a larger vulnerability score have more mutation opportunities. So that the entire fuzzing campaign develops in the direction of maximizing the vulnerability score of the execution trace. Although we have a direction, it is not a fixed direction (see Section V-E). Our dynamic direction places it in a middle ground between the coverage-guided algorithm and the traditional distance-based directed algorithm, which makes it possible to theoretically guarantee high vulnerability mining efficiency while jumping out of the local optimum problem. As shown in Table IV,

MODFuzz is able to find vulnerabilities in multiple driver modules and multiple vulnerability types.

V. EVALUATION

The implementation of MODFuzz is mainly divided into three parts: 1) kernel patch (about 1150 lines of C code); 2) static analysis (about 810 lines of Python code); and 3) fuzzing engine (about 1120 lines of C code besides original AFL). All relevant source codes are open-source. We evaluate MODFuzz by investigating the following research questions.

- 1) *RQ1*: Does ETP perform better than the algorithms of existing distance-based directed fuzzers? (see Section V-A).
- 2) *RQ2*: How helpful is the static vulnerability analysis to the overall performance of MODFuzz? (see Sections V-B and V-D).
- 3) *RQ3*: Is MODFuzz's execution overhead lower than other fuzzers for USB drivers? (see Section V-C).
- 4) *RQ4*: Does MODFuzz perform better than other fuzzers for USB drivers in vulnerability mining efficiency? (see Section V-D).

To answer these questions, we designed the following experiments. We conducted experiments in a VMware virtual machine with Ubuntu 20.04 installed. The underlying host has an Intel i7-12700K and 128 GB of RAM (DDR4 3600 MHz). We assigned eight logic cores and 110 GB of RAM to the Ubuntu VM to ensure that CPU core scheduling did not interfere with the experiments. We compared MODFuzz with the SOTA distance-based directed fuzzers (AFLGo and Hawkeye) and the SOTA USB driver fuzzers (USBfuzz and Syzkaller-USB). Since the fuzzers used for comparison are mainly implemented for the Linux kernel, and Section I has also introduced the prevalence of Linux kernel in IoT devices and the logical similarities of USB architectures under different IoT systems, we finally choose the Linux kernel as the experimental object. The differences between minor kernel version numbers are not big, and the Linux kernel has good forward compatibility. It makes little sense to test multiple kernel versions next to each other. We chose v5.6.19 and v6.4.10 as the kernel versions of our contrast experiments. We chose v5.6.19 due to compatibility issues (e.g., the limitation of USBfuzz's instrumentation solution makes it not compatible with the latest kernel versions). The v6.4.10 was the latest version when we started our experiments. In addition, we provided a case study to illustrate the advantages of MODFuzz compared to existing work in mining USB driver vulnerabilities.

A. Accuracy of ETP (RQ1)

The accuracy of the ETP is the basis of static vulnerability analysis. To answer RQ1, we design the following experiments to compare the performance of ETP and the SOTA distance calculation algorithms (i.e., AFLGo and Hawkeye's algorithms). Although AFLGo and Hawkeye have been established for several years, the latest distance-based directed fuzzers [30], [31], [32] continue to utilize their distance calculation algorithms. We modify AFLGo's

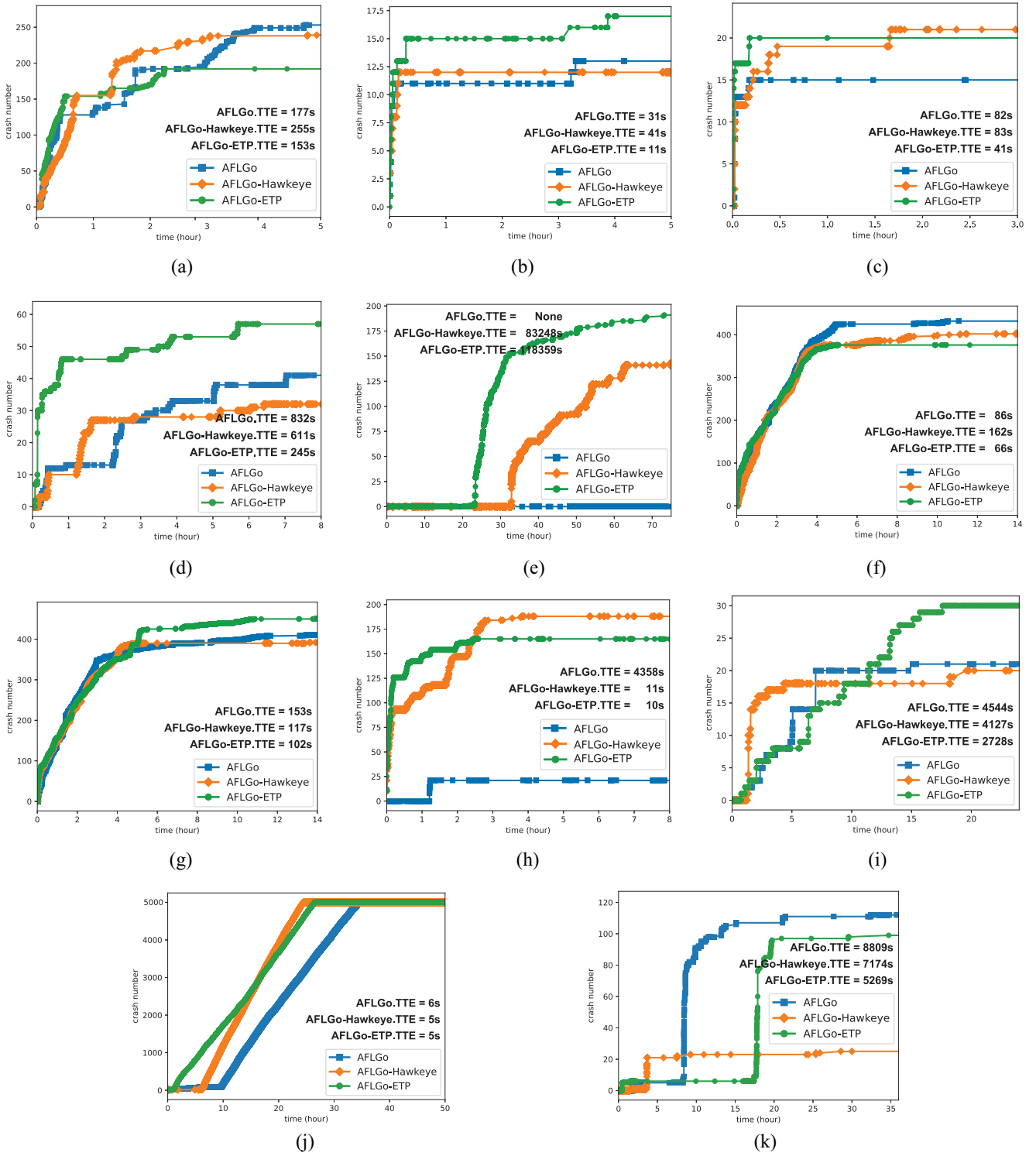


Fig. 8. Comparison of the performance of the original AFLGo, AFLGo-Hawkeye, and AFLGo-ETP in different vulnerability reproduction scenarios (provided by AFLGo project [64]). Lrzip-CVE-2017-8846, Lrzip-CVE-2018-11496, and objdump-CVE-2017-8392 are not included in the figure because they fail to produce any crash within 96 h. TTE for each scene is marked in each subfigure. (a) Cxxfilt-CVE-2016-4487. (b) Giflib-bugs-74. (c) Jasper-CVE-2015-5221. (d) Mjs-issues-57. (e) Mjs-issues-78. (f) Libming-CVE-2018-8807. (g) Libming-CVE-2018-8962. (h) Libxml2-ef709ce2. (i) LMS. (j) Palindrome. (k) KTY_Pretty_Printer.

source code, use ETP to replace AFLGo's original distance calculation algorithm and obtain AFLGo-ETP. Since Hawkeye is not open-source, our alternative is to implement adjacent-function distance augmentation of Hawkeye on

AFLGo's source code to get AFLGo-Hawkeye in the same way.

For the sake of fairness, we select 14 vulnerability reproduction scenarios provided by AFLGo project [64]. We set the

line of source code (provided by AFLGo project) that triggers the corresponding vulnerability as the fuzzing target in each scenario. However, in the scenarios of Irzip-CVE-2017-8846, Irzip-CVE-2018-11496, and objdump-CVE-2017-8392, all the three fuzzers (original AFLGo, AFLGo-Hawkeye and AFLGo-ETP) fail to produce any crash within 96 h. We speculate that this is due to inappropriate scenario settings provided by AFLGo project or the vulnerable versions of the target program have been fixed. TTE is used to measure the time interval until the first test input is generated that triggers a given error. We compare the TTE and the crash generation capabilities of the original AFLGo, AFLGo-Hawkeye, and AFLGo-ETP in the remaining 11 vulnerability reproduction scenarios. Since PDGF [29] has proved that a single traditional distance-based directed fuzzing will eventually converge, we define the state in each scenario where all the three fuzzers can not produce 5% of the total number of crashes in the last 5% of the total fuzzing time as the convergence of each fuzzing. We terminate fuzzing when convergence is reached in each scenario.

Our experiment results are shown in Fig. 8 (Fig. 8(e) is special because the original AFLGo failed to detect any crash within 72 h, while the other two fuzzers have reached convergence, so we terminated the fuzzing). In the early stage of fuzzing, AFLGo-ETP has the smallest TTE and generates more crashes than the original AFLGo and AFLGo-Hawkeye in all vulnerability reproduction scenarios. In the later stage of fuzzing, AFLGo-ETP's ability to generate crashes is not significantly stronger. However, from the perspective of measuring the accuracy of binary distance calculation, the application of ETP enables AFLGo-ETP to approach the predetermined targets more quickly, which can prove the accuracy of the ETP.

B. Effectiveness of Static Vulnerability Analysis (RQ2)

The purpose of static vulnerability analysis is to accurately calculate vulnerability scores for objective functions. However, it is impossible to know which kernel driver functions are objectively vulnerable in advance. In order to reasonably evaluate the effectiveness and accuracy of static vulnerability analysis (RQ2), we measure the time overhead of static vulnerability analysis and conduct experiments from the perspective of cross-validation of historical CVE disclosure information. We ran static vulnerability analysis three times in each kernel version, and the average results are shown in Table II. For both v5.6.19 and v6.4.10, function fragility analysis accounts for the largest proportion of time overhead. Compared with v5.6.19, the number of objective functions has increased significantly in v6.4.10, which is mainly due to the addition and changes of kernel drivers. Calculations for individual objective functions or cross-references do not differ significantly between the two kernel versions.

Fig. 9 illustrates the distribution of vulnerability scores for Linux v5.6.19 and Linux v6.4.10. The 99th percentiles (i.e., 99% of the scores in the set are below this value) of these scores are 0.597 and 0.527, respectively. Given that vulnerability scores directly influence seed mutation parameters, it can be deduced that a small but critical proportion of

TABLE II
TIME OVERHEAD OF STATIC VULNERABILITY ANALYSIS

Kernel Version	Size of S_{OF}	Average Analysis Time of S_{OF}	Size of S_{Xref}	Average Analysis Time of S_{Xref}	Total Analysis Time
5.6.19	21024	9.64s	1618	11.89s	221909s
6.4.10	30352	11.70s	1434	10.13s	369645s

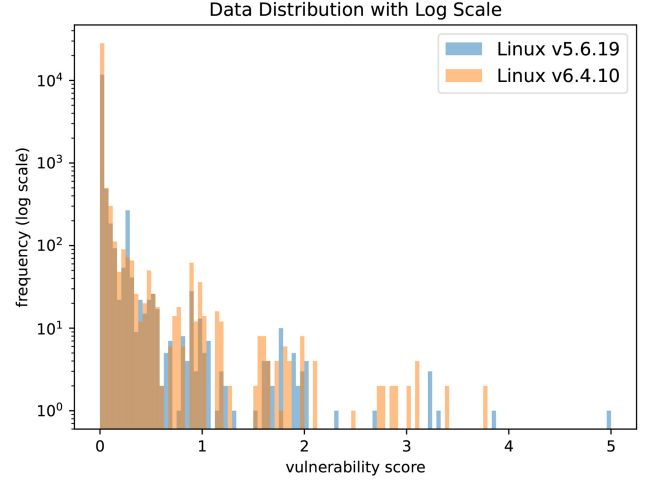


Fig. 9. Distribution of vulnerability score calculation results for the objective function sets of Linux v5.6.19 and Linux v6.4.10.

objective functions, characterized by high vulnerability scores, predominantly determines the direction of fuzzing. This result aligns with MODFuzz's design philosophy of concentrating fuzzing efforts on the most vulnerable areas.

In order to verify the effectiveness of static vulnerability analysis results, we use the vulnerabilities disclosed in CVE as ground truth. We select the top 100 and bottom 1000 objective functions with vulnerability scores and check whether they appear in the CVE list. As shown in Table III, among the top 100 objective functions of the two kernel versions, both versions have 11 objective functions (no overlap with S_{CVE}^{simp}) that have been publicly disclosed by CVE to have at least one vulnerability. Among the bottom 1000 objective functions of the two kernel versions, none of them have ever been exposed to have a vulnerability by CVE. We also calculated vulnerability scores for 115 functions that have been confirmed to be vulnerable. They are collected from the public information of USB driver-related vulnerabilities in the CVE list (by keyword search, CVE IDs that do not provide function names are filtered out). The results show that in both kernel versions, the average vulnerability score of the functions involved in the CVE list (0.441 in v5.6.19 and 0.570 in v6.4.10) is significantly higher than the average score in the objective function set (0.018 in v5.6.19, and 0.019 in v6.4.10). Although our vulnerability score can not be directly used as ground truth to evaluate a function, this cross-validation experiment proves the effectiveness of our static vulnerability analysis.

C. Execution Speed Comparison (RQ3)

In SOTA works on fuzzing USB drivers [7], [35], [42], [65], a necessary step is to emulate the entire kernel in

TABLE III
CORRESPONDENCE BETWEEN THE TOP 100 OBJECTIVE FUNCTIONS WITH VULNERABILITY SCORES AND THE CVE NUMBERS.
THIS TABLE DOES NOT LIST OTHER FUNCTIONS IN THE TOP 100 THAT DO NOT HAVE CORRESPONDING CVE NUMBERS

Linux v5.6.19			Linux v6.4.10		
Vulnerability Score	Objective Function Name	CVE Number	Vulnerability Score	Objective Function Name	CVE Number
4.990	tower_probe	CVE-2017-15102	3.108	usbtest_disconnect	CVE-2020-15393
3.228	usbtest_disconnect	CVE-2020-15393	2.489	tower_probe	CVE-2017-15102
2.329	create_fixed_stream_quirk	CVE-2016-2184	1.724	aiptek_probe	CVE-2015-7515
1.750	gtco_probe	CVE-2016-2187	1.654	em28xx_usb_probe	CVE-2022-3239
1.686	aiptek_probe	CVE-2015-7515	1.624	snd_usb_caiaq_midi_init	CVE-2011-0712
1.500	ttusbdecfe_dvbs_diseqc_send_master_cmd	CVE-2014-8884	1.500	ttusbdecfe_dvbs_diseqc_send_master_cmd	CVE-2014-8884
1.078	usb_get_bos_descriptor	CVE-2017-16535	1.152	snd_usb_caiaq_audio_init	CVE-2011-0712
0.871	flexcop_usb_probe	CVE-2019-15291	1.015	imon_probe	CVE-2017-16537
0.844	af9005_identify_state	CVE-2019-18809	0.999	usbip_sockfd_store	CVE-2021-29265
0.833	cpia2_exit	CVE-2019-19966	0.965	dwc3_pci_probe	CVE-2019-18813
0.805	usb_destroy_configuration	CVE-2017-17558	0.831	iowarrior_probe	CVE-2016-2188

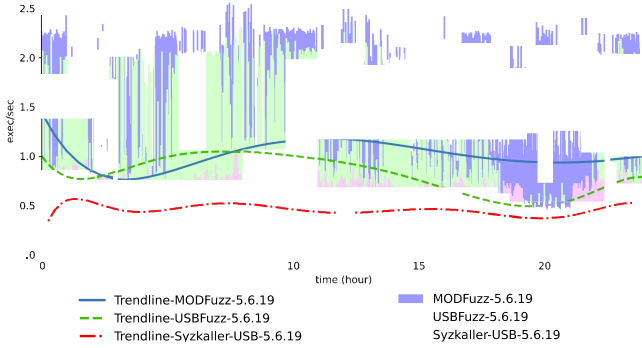


Fig. 10. Execution speed comparison results obtained after 24 h of fuzzing. The curves represent the trend line obtained by polynomial fitting.

a VM. This results in the execution speed of the kernel driver fuzzing (usually below 20 exec/s) being much slower than user-mode program fuzzing (usually can reach thousands exec/s). Considering the open-source factor and the relevance to this work, we choose Syzkaller-USB (Syzkaller's USB-fuzzer extension) and USBFuzz for execution speed comparison to answer RQ3. In order to ensure the fairness of the experiment, we use the same target kernel (v5.6.19), the same instrumentation scope (only instrument kernel modules related to USB drivers), and the same hardware environment for 24 h fuzzing.

The results are as shown in Fig. 10. In experiments on Linux kernel v5.6.19, the average execution speed of MODFuzz (0.96 exec/s) was improved by approximately 109% and 19% compared to Syzkaller-USB (0.46 exec/s) and USBFuzz (0.81 exec/s), respectively. However, the performance of fuzzing may vary across different mutation stages (e.g., bitflip and havoc require different time costs, Fig. 10 demonstrates evident periodicity). To ensure a more equitable assessment of execution speed, we conducted a dry-run comparison test. Notably, the dry-run stage excludes any seed mutation operations, thereby eliminating the impact of mutation stages on execution speed. Since Syzkaller-USB does not support dry-run mode, we only evaluated the average dry-run execution time of USBFuzz and MODFuzz with the same initial seed set (comprised of 695 USB descriptors provided by USBFuzz) in kernel v5.6.19. The average dry-run time for MODFuzz was 565 ms (1.77 exc/s), while USBFuzz recorded an average of 641 ms (1.56 exc/s). Both USBFuzz and MODFuzz

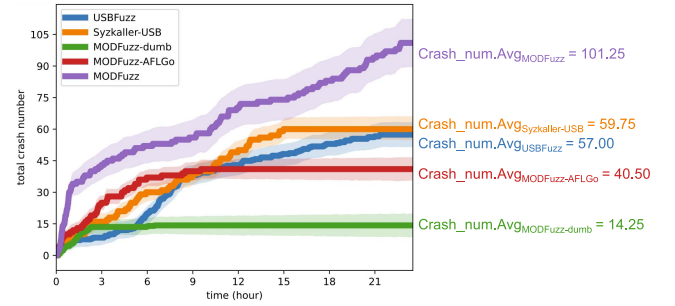


Fig. 11. Comparison of the number of total crashes generated by different fuzzers within 24 h in Linux kernel v5.6.19.

leverage QEMU emulation for executing the drivers in the target kernel. The variation in performance during the dry-run phase primarily stems from the kernel binary's instrumentation method. MODFuzz adopts a lightweight instrumentation approach, resulting in an overall execution speed improvement of approximately 14%.

D. Vulnerability Mining Comparison (RQ2 and RQ4)

To answer RQ2 and RQ4, we evaluate the vulnerability mining capabilities by comparing the number of crashes generated by five tools (USBFuzz [7], Syzkaller-USB [35], MODFuzz-dumb, MODFuzz-AFLGo and MODFuzz) in the same kernel version within 24 h fuzzing campaign (each fuzzer is tested 8 times). Considering that USBFuzz does not support the latest kernel version, we used kernel v5.6.19 as the test target. MODFuzz-dumb refers to the fuzzer obtained by setting the vulnerability scores of all objective functions to 0. That prevents it from benefiting from static vulnerability analysis. MODFuzz-AFLGo refers to the fuzzer obtained by setting the functions with the top 100 vulnerability scores as the fixed fuzzing targets by AFLGo's algorithm. It is equivalent to an AFLGo capable of fuzzing USB drivers.

As shown in Fig. 11, MODFuzz is able to generate crashes faster in the earliest stage and ultimately generates the most crashes (about 78% improvement compared to USBFuzz and about 69% improvement compared to Syzkaller-USB). The crashes generation capability of MODFuzz is increased by 710% compared with MODFuzz-Dumb, which indicates that static vulnerability analysis greatly improves the overall performance of MODFuzz (RQ2). Both MODFuzz

TABLE IV
KERNEL BUGS FOUND BY MODFUZZ IN LINUX KERNEL V6.4.10. THEY
HAVE ALL BEEN REPORTED TO THE DEVELOPMENT TEAM

Vulnerability Function	Module	Vulnerability Type	Vulnerability ID
usb_giveback_urb	USB Core	Denial of Service	CVE-2023-4010
dvb_usb_device_init	DVB-USB	Heap Overflow	CNVD-ZC-2024-1750
pvr2_send_request_ex	PVR-USB2	Denial of Service	CNVD-ZC-2024-1752
usbhid_probe	USB HID	Denial of Service	CNVD-2023-71732
au0828_usb_probe	AU0828	Use-After-Free	CNVD-2024-23802
usb_hcd_map_urb_for_dma	USB HCD	Stack Overflow	CNVD-ZC-2024-1755

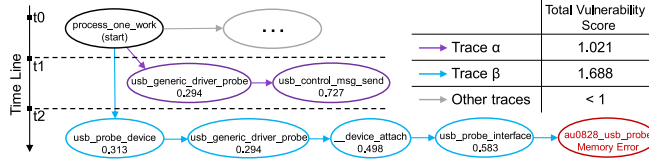


Fig. 12. Process of MODFuzz discovering the UAF bug in `au0828_usb_probe`. The number below the function name is its vulnerability score. Irrelevant functions and execution traces are omitted.

and MODFuzz-AfLGo have fast crash generation speed in the early stage. But MODFuzz-AFLGo, as a distance-based directed fuzzer, converges early and can not generate more crashes in the later stage of fuzzing. This shows that the dynamic fuzzing direction of MODFuzz can avoid the local optimum problem of traditional distance-based fuzzing to a certain extent.

Zero-Day Vulnerabilities: We ran MODFuzz in Linux kernel v6.4.10 for 72 h and generated a total of 286 crashes. After manual verification, we got 49 bugs. After removing duplicates and known bugs, as shown in Table IV, 6 of the 49 bugs were unique and unknown before. We reported them and received the CVE ID and acknowledgment from Red Hat.

E. Case Study: Why Is the Direction of MODFuzz Dynamic?

As shown in Fig. 12, MODFuzz calculated the total vulnerability scores of executed traces during fuzzing process. In t_1 , MODFuzz triggered the Trace α and got the larger score (1.021) than other previous traces. From t_1 to t_2 , more fuzzing resources were allocated to the seed that triggered Trace α . In t_2 , MODFuzz discovered Trace β with a higher score (1.688), and then more fuzzing resources were dynamically transferred to the seed that triggered the Trace β , which finally resulted in the UAF in `au0828_usb_probe`. This bug was caused by the reuse of freed member pointer in the structure `usb_interface`, which was created in `usb_generic_driver_probe`, freed and reused in `au0828_usb_probe`.

This bug arises from a logical flaw in global data processing between multiple functions involving sensitive memory operations in the same trace. Unfortunately, when the USB drivers handle USB devices, scenarios like this happen frequently. MODFuzz's strategy of using the total vulnerability score of the trace to guide fuzzing direction dynamically makes it easier to find deeper bugs caused by improper handling of multiple functions (such bugs can not be found by only analyzing each individual function in the trace). MODFuzz can dynamically adjust the fuzzing direction based on the vulnerability score of

the execution trace, which is the capability currently lacking in SOTA distance-based directed fuzzers.

VI. DISCUSSION

In this section, we discuss some limitations of the current work as well as the future work.

A. Limitations

Static Analysis Overhead: MODFuzz relies on fine-grained static analysis to calculate the vulnerability scores of multiple objective functions. As can be seen from Table II, static analysis is time-consuming. Compared with other fuzzers that can be started almost immediately, MODFuzz needs to perform static analysis on the target before fuzzing. This makes MODFuzz not time-efficient in short-term fuzzing. However, this disadvantage will be alleviated in fuzzing with a time unit of weeks.

False Positive Crashes: Since the Linux kernel has a relatively complete fault tolerance mechanism, the crash of a single kernel thread does not result in significantly perceptible changes to the overall system. The current crash detection method (i.e., capture thread crashes by keyword matching in the kernel log) is prone to false positive crashes. This is also the problem that plagues the current SOTA USB fuzzers (Syzkaller, USBFuzz, ReUSB, etc.). We plan to achieve more accurate detection of kernel thread crashes through customized instrumentation of kernel exception handling functions in our next work.

B. Future Work

Adapt to Specific IoT Devices: MODFuzz's current experiments are conducted in a simulated Linux kernel. However, the framework of MODFuzz can be migrated to fuzz specific IoT devices. USB interfaces are very popular in IoT devices. After simulating the firmware of the target IoT device and using the MODFuzz's algorithm for fuzzing, vulnerabilities in the USB module of the target IoT device can be effectively discovered.

Migrate to Other Fuzzing Targets: MODFuzz focuses on fuzzing the USB driver framework. By adjusting the instrumentation scope and objective function set, MODFuzz can also be easily migrated to other fuzzing scenarios (e.g., PCI device drivers). Our multiobjective directed algorithm can also be migrated to fuzz user space general-purpose software.

Resolve Branch Probability Through Execution Records: The probabilities of different branches have some regularity. We plan to record the results of each branch through kernel instrumentation in advance. After executing the target kernel multiple times, we can get the probability of each branch by the statistics of real execution data. The branches' probabilities can be utilized as heuristic information when calculating the ETP in the later static analysis.

VII. CONCLUSION

We present MODFuzz, a multiobjective directed fuzzer for USB drivers. We apply directed fuzzing to USB drivers

TABLE V
DEFAULT INSTRUMENTATION SCOPE (IN LINUX V6.4.10) FOR MODFUZZ

Folder Directory	Number of Functions Included
lib/	4561
sound/usb/	1214
drivers/usb/	12163
drivers/media/usb/	3594
drivers/hid/	2476
drivers/input/	5121
drivers/nfc/	668
drivers/net/usb/	1263
drivers/staging/greybus/	601
drivers/staging/most/net/	17
drivers/staging/most/video/	26

for the first time. Unlike existing distance-based directed fuzzers, MODFuzz does not require manual target specification. MODFuzz builds a vulnerability score set by performing static vulnerability analysis on multiple objective functions and makes the entire fuzzing process develop in a dynamic direction of maximizing the vulnerability scores of execution traces. Evaluations on MODFuzz show that it outperforms current SOTA works in crash generation and execution speed. MODFuzz discovered six unknown bugs in Linux v6.4.10. They are distributed at various levels of the USB driver framework and belong to different vulnerability types.

APPENDIX

DEFAULT INSTRUMENTATION SCOPE FOR MODFUZZ

See Table V.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their insightful comments.

REFERENCES

- [1] "USB implementers forum." 2023. [Online]. Available: <https://www.usb.org/>
- [2] "USB/IP Project." 2011. [Online]. Available: <https://usbip.sourceforge.net/>
- [3] D. Paradis and B. Segee, "Using USBIP to create a portable remote USB hub," in *Proc. Int. Conf. Internet Comput. (ICOMP)*, 2017, pp. 48–51.
- [4] "Usbredir." SPICE space project. 2024. [Online]. Available: <https://www.spice-space.org/usbredir.html>
- [5] S. Liu, G. Peng, H. Zeng, and J. Fu, "A survey on the evolution of fileless attacks and detection techniques," *Comput. Security*, vol. 137, Feb. 2024, Art. no. 103653. [Online]. Available: <https://doi.org/10.1016/j.cose.2023.103653>
- [6] X. Yang, G. Peng, D. Zhang, Y. Gao, and C. Li, "PowerDetector: Malicious powershell script family classification based on multi-modal semantic fusion and deep learning," *China Commun.*, vol. 20, no. 11, pp. 202–224, Nov. 2023.
- [7] H. Peng and M. Payer, "USBfuzz: A framework for fuzzing USB drivers by device emulation," in *Proc. 29th USENIX Security Symp. (USENIX Security)*, 2020, pp. 2559–2575.
- [8] M. Zalewski. "American fuzzy lop." 2020. [Online]. Available: <https://lcamtuf.coredump.cx/afl/>
- [9] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based grey-box fuzzing as Markov chain," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2016, pp. 1032–1043.
- [10] J. Hur, S. Song, D. Kwon, E. Baek, J. Kim, and B. Lee, "DifuzzRTL: Differential fuzz testing to find CPU bugs," in *Proc. IEEE Symp. Security Privacy (SP)*, 2021, pp. 1286–1303.
- [11] C. Myung, G. Lee, and B. Lee, "MundoFuzz: Hypervisor fuzzing with statistical coverage testing and grammar inference," in *Proc. 31st USENIX Security Symp. (USENIX Security)*, 2022, pp. 1257–1274.
- [12] Z. Shen, R. Roongta, and B. Dolan-Gavitt, "Drifuzz: Harvesting bugs in device drivers from golden seeds," in *Proc. 31st USENIX Security Symp. (USENIX Security)*, 2022, pp. 1275–1290.
- [13] T. Scharnowski et al., "Fuzzware: Using precise MMIO modeling for effective firmware fuzzing," in *Proc. 31st USENIX Security Symp. (USENIX Security)*, 2022, pp. 1239–1256.
- [14] A. Bulekov, B. Das, S. Hajnoczi, and M. Egele, "MORPHUZZ: Bending (input) space to fuzz virtual devices," in *Proc. 31st USENIX Security Symp. (USENIX Security)*, 2022, pp. 1221–1238.
- [15] J. Wang, C. Song, and H. Yin, "Reinforcement learning-based hierarchical seed scheduling for greybox fuzzing," in *Proc. 28th Annu. Netw. Distrib. Syst. Security Symp.*, 2021, pp. 1–17.
- [16] X. Xie et al., "DeepHunter: A coverage-guided fuzz testing framework for deep neural networks," in *Proc. 28th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, 2019, pp. 146–157.
- [17] W. Luo, D. Chai, X. Ruan, J. Wang, C. Fang, and Z. Chen, "Graph-based fuzz testing for deep learning inference engines," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, 2021, pp. 288–299.
- [18] F. Gao, Y. Wang, L. Situ, and L. Wang, "Deep learning-based hybrid fuzz testing," *Int. J. Softw. Inform.*, vol. 11, no. 3, pp. 1–21, 2021.
- [19] M. Böhme, V. Pham, M. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2017, pp. 2329–2344. [Online]. Available: <https://doi.org/10.1145/3133956.3134020>
- [20] M. Nguyen, S. Bardin, R. Bonichon, R. Groz, and M. Lemerre, "Binary-level directed fuzzing for use-after-free vulnerabilities," in *Proc. 23rd Int. Symp. Res. Attacks, Intrusions Defenses*, 2020, pp. 47–62. [Online]. Available: <https://www.usenix.org/conference/raid2020/presentation/nguyen>
- [21] T. Wang, T. Wei, G. Gu, and W. Zou, "TaintScope: A checksum-aware directed fuzzing tool for automatic software vulnerability detection," in *Proc. 31st IEEE Symp. Security Privacy*, 2010, pp. 497–512. [Online]. Available: <https://doi.org/10.1109/SP.2010.37>
- [22] G. Lee, W. Shim, and B. Lee, "Constraint-guided directed grey-box fuzzing," in *Proc. 30th USENIX Security Symp.*, 2021, pp. 3559–3576. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/lee-gwangmu>
- [23] P. Zong, T. Lv, D. Wang, Z. Deng, R. Liang, and K. Chen, "FuzzGuard: Filtering out unreachable inputs in directed grey-box fuzzing through deep learning," in *Proc. 29th USENIX Security Symp.*, 2020, pp. 2255–2269. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity20/presentation/zong>
- [24] V. Ganesh, T. Leek, and M. Rinard, "Taint-based directed whitebox fuzzing," in *Proc. IEEE 31st Int. Conf. Softw. Eng.*, 2009, pp. 474–484.
- [25] H. Chen et al., "Hawkeye: Towards a desired directed grey-box fuzzer," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2018, pp. 2095–2108. [Online]. Available: <https://doi.org/10.1145/3243734.3243849>
- [26] H. Huang, Y. Guo, Q. Shi, P. Yao, R. Wu, and C. Zhang, "BEACON: Directed grey-box fuzzing with provable path pruning," in *Proc. 43rd IEEE Symp. Security Privacy*, 2022, pp. 36–50. [Online]. Available: <https://doi.org/10.1109/SP46214.2022.9833751>
- [27] H. Liang, X. Yu, X. Cheng, J. Liu, and J. Li, "Multiple targets directed greybox fuzzing," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 1, pp. 325–339, Jan./Feb. 2024.
- [28] H. Huang, P. Yao, H.-C. Chiu, Y. Guo, and C. Zhang, "Titan: Efficient multi-target directed greybox fuzzing," in *Proc. IEEE Symp. Security Privacy (SP)*, 2024, pp. 1849–1864.
- [29] Y. Zhang, Y. Liu, J. Xu, and Y. Wang, "Predecessor-aware directed greybox fuzzing," in *Proc. IEEE Symp. Security Privacy (SP)*, May 2024, pp. 1884–1900. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/SP54263.2024.00040>
- [30] T. E. Kim, J. Choi, K. Heo, and S. K. Cha, "DAFL: Directed grey-box fuzzing guided by data dependency," in *Proc. 32nd USENIX Security Symp.*, 2023, pp. 4931–4948. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/kim-tae-eun>
- [31] Z. Du, Y. Li, Y. Liu, and B. Mao, "Windranger: A directed greybox fuzzer driven by deviation basic blocks," in *Proc. 44th IEEE/ACM 44th Int. Conf. Softw. Eng.*, 2022, pp. 2440–2451. [Online]. Available: <https://doi.org/10.1145/3510003.3510197>
- [32] C. Luo, W. Meng, and P. Li, "SelectFuzz: Efficient directed fuzzing with selective path exploration," in *Proc. 44th IEEE Symp. Security Privacy*, 2023, pp. 2693–2707. [Online]. Available: <https://doi.org/10.1109/SP46215.2023.10179296>
- [33] Y. Wang et al., "Not all coverage measurements are equal: Fuzzing by coverage accounting for input prioritization," in *Proc. 27th Annu. Netw. Distrib. Syst. Security Symp.*, 2020, pp. 1–17.

- [34] B. Elad. "Linux statistics 2024 by market share, usage data, number of users and facts." 2024. [Online]. Available: <https://www.enterpriseappstoday.com/stats/linux-statistics.html>
- [35] (Google, Mountain View, CA, USA). *External USB Fuzzing for Linux Kernel*. 2023. [Online]. Available: https://github.com/google/syzkaller/blob/master/docs/linux/external_fuzzing_usb.md
- [36] (Honeywell, Charlotte, NC, USA). *The 2024 USB Threat Report from Honeywell GARD*. 2024. [Online]. Available: <https://hcnnews.honeywell.com/CYB-2024-Threat-Report-LP>
- [37] D. Kushner. "The real story of Stuxnet." 2013. [Online]. Available: <https://spectrum.ieee.org/the-real-story-of-stuxnet>
- [38] "MODFuzz source code and related data." 2024. [Online]. Available: <https://anonymous.4open.science/r/MODFuzz-B5F2/README.md>
- [39] (USB Implementers Forum, Beaverton, OR, USA). *USB4® Specification V2.0*. 2023. [Online]. Available: <https://www.usb.org/document-library/usb4r-specification-v20>
- [40] K. Kim, S. Kim, K. R. B. Butler, A. Bianchi, R. Kennell, and D. J. Tian, "Fuzz the power: Dual-role state guided black-box fuzzing for USB power delivery," in *Proc. 32nd USENIX Security Symp. (USENIX Security)*, Aug. 2023, pp. 5845–5861. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/kim-kyungtae>
- [41] R. van Tonder and H. A. Engelbrecht, "Lowering the USB fuzzing barrier by transparent two-way emulation," in *Proc. 8th USENIX Workshop Offensive Technol.*, 2014, pp. 1–8. [Online]. Available: <https://www.usenix.org/conference/woot14/workshop-program/presentation/van-tonder>
- [42] J. Jang, M. Kang, and D. Song, "ReUSB: Replay-guided USB driver fuzzing," in *Proc. 32nd USENIX Security Symp.*, 2023, pp. 1–19. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/jang>
- [43] K. Kim et al., "Fuzzusb: Hybrid stateful fuzzing of USB gadget stacks," in *Proc. IEEE Symp. Security Privacy (SP)*, 2022, pp. 2212–2229.
- [44] "QEMU: A generic and open source machine emulator and virtualizer." 2025. [Online]. Available: <https://www.qemu.org/>
- [45] S. Österlund, K. Razavi, H. Bos, and C. Giuffrida, "ParmeSan: Sanitizer-guided greybox fuzzing," in *Proc. 29th USENIX Security Symp. (USENIX Security)*, 2020, pp. 2289–2306.
- [46] (MITRE Corp., McLean, VA, USA). *Common Vulnerabilities and Exposures*. 2024. [Online]. Available: <https://cve.mitre.org/>
- [47] "IDA pro." Hex-Rays. 2025. [Online]. Available: <https://hex-rays.com/ida-pro/>
- [48] (MITRE Corp., McLean, VA, USA). *CVE-2023-2194*. 2023. [Online]. Available: <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2023-2194>
- [49] "KCOV: Code coverage for fuzzing." Kernel development community. 2025. [Online]. Available: <https://docs.kernel.org/dev-tools/kcov.html>
- [50] C. Macdonell. "Device specification for inter-VM shared memory device." 2020. [Online]. Available: <https://github.com/qemu/qemu/blob/master/docs/specs/ivshmem-spec.txt>
- [51] D. Brumley. "The hard things about analyzing 1's and 0's." 2020. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/autodraft-83/>
- [52] Y. Shoshitaishvili et al., "SOK: (State of) the art of war: Offensive techniques in binary analysis," in *Proc. IEEE Symp. Security Privacy (SP)*, 2016, pp. 138–157.
- [53] Y. Shoshitaishvili, R. Wang, C. Hauser, C. Kruegel, and G. Vigna, "Firmallice—Automatic detection of authentication bypass vulnerabilities in binary firmware," in *Proc. NDSS*, 2015, pp. 1–15.
- [54] J. Geldenhuys, M. B. Dwyer, and W. Visser, "Probabilistic symbolic execution," in *Proc. Int. Symp. Softw. Test. Anal.*, 2012, pp. 166–176. [Online]. Available: <https://doi.org/10.1145/2338965.2336773>
- [55] S. Saha, M. Downing, T. Brennan, and T. Bultan, "PREACH: A heuristic for probabilistic reachability to identify hard to reach statements," in *Proc. 44th IEEE/ACM 44th Int. Conf. Softw. Eng.*, 2022, pp. 1706–1717. [Online]. Available: <https://doi.org/10.1145/3510003.3510227>
- [56] S. Lee and M. Böhme, "Statistical reachability analysis," in *Proc. 31st ACM Joint Eur. Symp. Conf. Found. Softw. Eng.*, 2023, pp. 326–337. [Online]. Available: <https://doi.org/10.1145/3611643.3616268>
- [57] "Basic concept or 'what is an URB?'" 2025. [Online]. Available: <https://docs.kernel.org/driver-api/usb/URB.html>
- [58] (MITRE Corp., McLean, VA, USA). *CVE-2023-3077*. 2023. [Online]. Available: <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2022-3077>
- [59] Y. Chen and X. Xing, "SLAKE: Facilitating slab manipulation for exploiting vulnerabilities in the Linux kernel," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2019, pp. 1707–1722.
- [60] W. Chen, X. Zou, G. Li, and Z. Qian, "KOOBE: Towards facilitating exploit generation of kernel out-of-bounds write vulnerabilities," in *Proc. 29th USENIX Security Symp. (USENIX Security)*, 2020, pp. 1093–1110.
- [61] Z. Lin et al., "GREBE: Unveiling exploitation potential for Linux kernel bugs," in *Proc. IEEE Symp. Security Privacy (SP)*, 2022, pp. 2078–2095.
- [62] "x86-64 calling conventions." Wikipedia. 2025. [Online]. Available: https://en.wikipedia.org/wiki/X86_calling_conventions#x86-64_calling_conventions
- [63] "Facedancer21." GoodFET. 2021. [Online]. Available: <https://goodfet.sourceforge.net/hardware/facedancer21/>
- [64] "AFLGo's test scenarios." 2023. [Online]. Available: <https://github.com/aflgo/aflgo/tree/master/examples>
- [65] Y. Xu, H. Sun, J. Liu, Y. Shen, and Y. Jiang, "SATURN: Host-gadget synergistic USB driver fuzzing," in *Proc. IEEE Symp. Security Privacy (SP)*, May 2024, pp. 4646–4660. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/SP54263.2024.00051>